МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования
САМАРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Институт автоматики и информационных технологий
Кафедра «Информатика и вычислительная техника»

ОТЧЕТ

о выполнении лабораторной работы №2

по дисциплине Системное программное обеспечение

на тему Монитор процессов и потоков

Преподаватель	доцент			К.В. Портнов
	(должность)	(подпись)	(дата)	(инициалы, фамилия)
	ст. преподаватель			С.А. Федосов
	(должность)	(подпись)	(дата)	(инициалы, фамилия)
Студенты	3-ИАИТ-119			П.М. Марченко
	(группа)	(подпись)	(дата)	(инициалы, фамилия)
	3-ИАИТ-119			И.Д. Царегородцев
	(группа)	(подпись)	(дата)	(инициалы, фамилия)

Самара 2026 г.

Цель работы: практическое знакомство с использованием функций Win32 API для получения информации о процессах, потоках и используемых ими ресурсах ОС Windows в консольном приложении на языке C++.

Задание.

Общее задание (для всех бригад):

1) вывести на экран список процессов, выполняющихся в системе;

2) для выбранного процесса, идентификатор которого вводится с клавиатуры, вывести сведения о его приоритете и количестве потоков;

3) для выбранного прикладного процесса, идентификатор которого вводится с клавиатуры, вывести время его работы в режиме ядра и в режиме пользователя.

Индивидуальное задание (бригада № 10):

1) вывести информацию о загрузке процессора в процентах;

2) вывести список имен прикладных процессов, использовавших более M байт оперативной памяти; значение M задается с клавиатуры при запуске программы.

Результаты сравнить с данными Диспетчера задач.

Работа выполнена в среде Microsoft Visual Studio на языке C++. Создано решение Lab2 из трех консольных приложений: SPO_Lr2 (общее задание), SPO_Lr2_CpuLoad (индивидуальное задание 1) и SPO_Lr2_Memory (индивидуальное задание 2). Использованы только заголовочные файлы и библиотеки, рассмотренные в методических указаниях: windows.h, tlhelp32.h, psapi.h (psapi.lib), pdh.h (pdh.lib), iostream, cstdio, clocale, conio.h.

1. Использованные функции Win32 API.

Таблица 1 – Функции Win32 API, использованные в работе

Функция	Заголовок	Назначение
CreateToolhelp32Snapshot	tlhelp32.h	Создает снимок процессов (TH32CS_SNAPPROCESS) или потоков (TH32CS_SNAPTHREAD) системы
Process32First, Process32Next	tlhelp32.h	Извлекают из снимка сведения о первом и следующем процессе в структуру PROCESSENTRY32
Thread32First, Thread32Next	tlhelp32.h	Извлекают из снимка сведения о потоках в структуру THREADENTRY32
OpenProcess	windows.h	Возвращает дескриптор процесса по его идентификатору

Функция	Заголовок	Назначение
CloseHandle	windows.h	Закрывает дескриптор снимка или процесса
GetPriorityClass	windows.h	Возвращает класс приоритета процесса
GetProcessTimes	windows.h	Возвращает время создания процесса, время работы в режиме ядра и в режиме пользователя (FILETIME)
FileTimeToLocalFileTime	windows.h	Преобразует время UTC в местное время
FileTimeToSystemTime	windows.h	Преобразует FILETIME в структуру SYSTEMTIME
ProcessIdToSessionId	windows.h	Возвращает номер сеанса, в котором выполняется процесс (сеанс 0 — службы и системные процессы)
GetProcessMemoryInfo	psapi.h	Возвращает сведения об использовании памяти процессом (PROCESS_MEMORY_COUNTERS)
PdhOpenQuery	pdh.h	Создает запрос для сбора данных счетчиков производительности
PdhMakeCounterPath	pdh.h	Формирует путь к счетчику из элементов (PDH_COUNTER_PATH_ELEMENTS)
PdhAddEnglishCounter	pdh.h	Добавляет счетчик в запрос по английскому имени (независимо от языка Windows)
PdhCollectQueryData	pdh.h	Выполняет замер значений счетчиков запроса
PdhGetFormattedCounterValue	pdh.h	Возвращает значение счетчика в заданном формате (PDH_FMT_DOUBLE)
PdhRemoveCounter, PdhCloseQuery	pdh.h	Удаляют счетчик и закрывают запрос
Sleep	windows.h	Приостанавливает выполнение потока на заданное время
GetLocalTime	windows.h	Возвращает текущие местные дату и время
GetLastError	windows.h	Возвращает код ошибки последней функции

2. Общее задание 1. Список процессов.

Алгоритм (функция PrintProcessList):

1) создать снимок процессов:
   CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);

2) присвоить полю dwSize структуры PROCESSENTRY32 размер структуры;

3) получить первый процесс функцией Process32First, затем в цикле получать следующие функцией Process32Next, пока она не вернет FALSE;

4) для каждого процесса вывести PID (th32ProcessID), PID родителя (th32ParentProcessID), количество потоков (cntThreads), базовый приоритет (pcPriClassBase) и имя (szExeFile);

5) закрыть дескриптор снимка и вывести общее количество процессов.

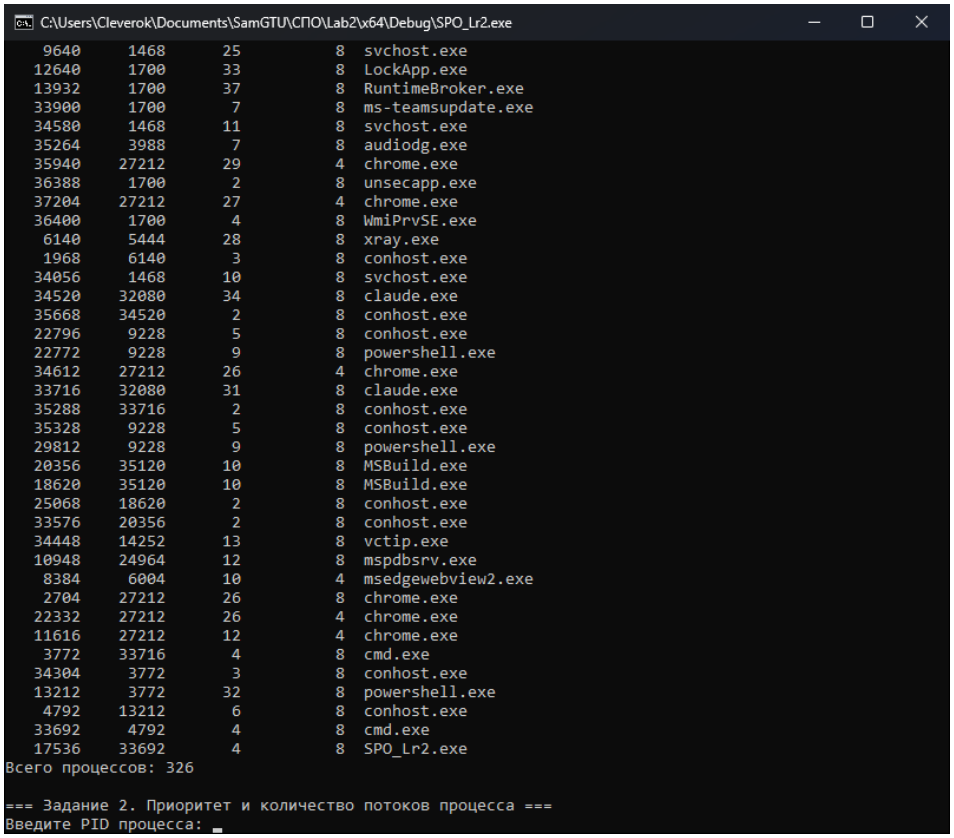


Рисунок 1 – Окончание списка процессов (программа SPO_Lr2)

Комментарий. Программа вывела 326 процессов (на рисунке 1 – окончание списка). Список содержит как системные процессы и службы (svchost.exe, WmiPrvSE.exe), так и прикладные (chrome.exe, powershell.exe). По полю PPID видно дерево процессов: например, conhost.exe и powershell.exe запущены процессом cmd.exe. Количество процессов при повторных запусках немного меняется, так как процессы постоянно создаются и завершаются.

3. Общее задание 2. Приоритет и количество потоков процесса.

Алгоритм (функции FindProcess и PrintPriorityAndThreads):

1) ввести PID процесса с клавиатуры;

2) найти процесс в снимке процессов, перебирая его функциями Process32First / Process32Next;

3) вывести базовый приоритет (pcPriClassBase) и количество потоков (cntThreads);

4) открыть процесс функцией OpenProcess и получить класс приоритета функцией GetPriorityClass;

5) создать снимок потоков (TH32CS_SNAPTHREAD), перебрать его функциями Thread32First / Thread32Next и вывести потоки, у которых th32OwnerProcessID равен PID, с их приоритетами (tpBasePri).

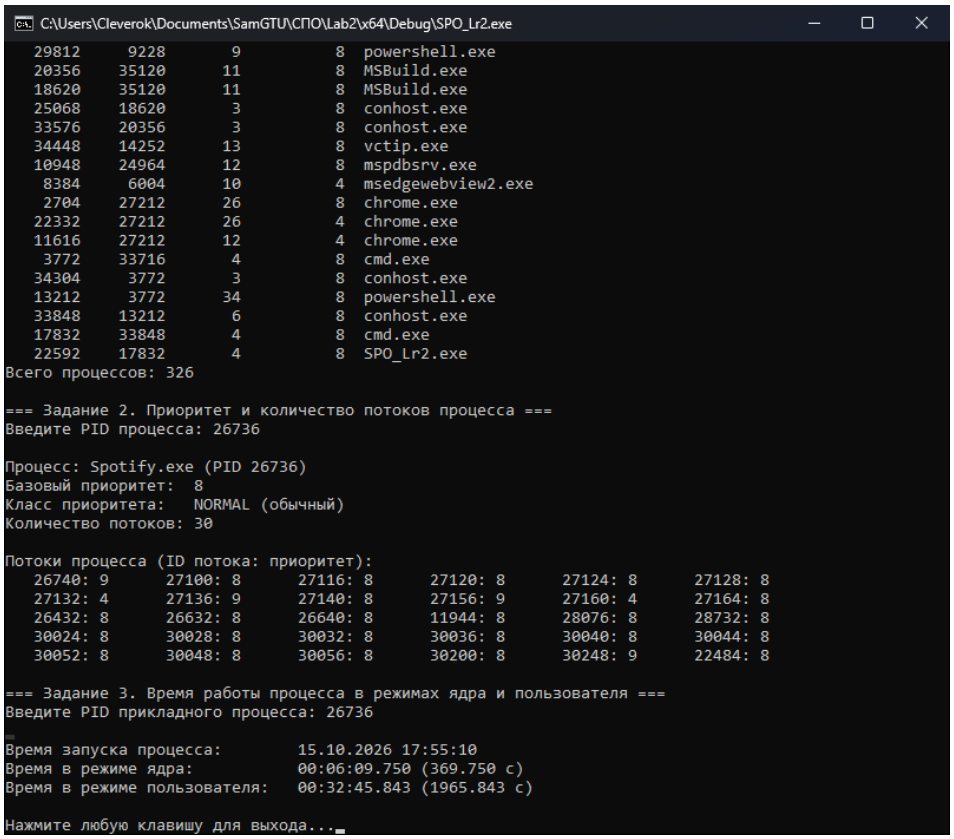


Рисунок 2 – Приоритет, потоки и время работы процесса Spotify.exe

Комментарий. Для процесса Spotify.exe (PID 26736) получены базовый приоритет 8 и класс NORMAL, количество потоков – 30 (рисунок 2). Те же значения выдает команда PowerShell Get-Process (BasePriority = 8, PriorityClass = Normal, 30 потоков). Приоритеты отдельных потоков (4, 8, 9) отличаются от базового приоритета процесса, так как приложение задает своим потокам относительный приоритет.

4. Общее задание 3. Время работы процесса в режимах ядра и пользователя.

Алгоритм (функция PrintProcessTimes):

1) ввести PID прикладного процесса с клавиатуры;

2) получить дескриптор процесса функцией OpenProcess с правом PROCESS_QUERY_LIMITED_INFORMATION;

3) вызвать GetProcessTimes и получить время создания, завершения, работы в режиме ядра и в режиме пользователя в структурах FILETIME;

4) время создания перевести в местное время функциями FileTimeToLocalFileTime и FileTimeToSystemTime;

5) длительности перевести из 100-наносекундных интервалов в миллисекунды и вывести в формате чч:мм:сс.мс.

Комментарий. Для Spotify.exe получены время в режиме ядра 00:06:09.750 и в режиме пользователя 00:32:45.843 (рисунок 2); Get-Process показывает 00:06:09.75 и 00:32:45.84 – значения совпадают. Время работы в режиме пользователя больше, так как основную работу приложение выполняет в своем коде, а не в системных вызовах.

Для системных процессов функция OpenProcess завершается с ошибкой 5 (доступ запрещен), поэтому задание выполняется для прикладных процессов. Программа сообщает об этом (рисунок 3).

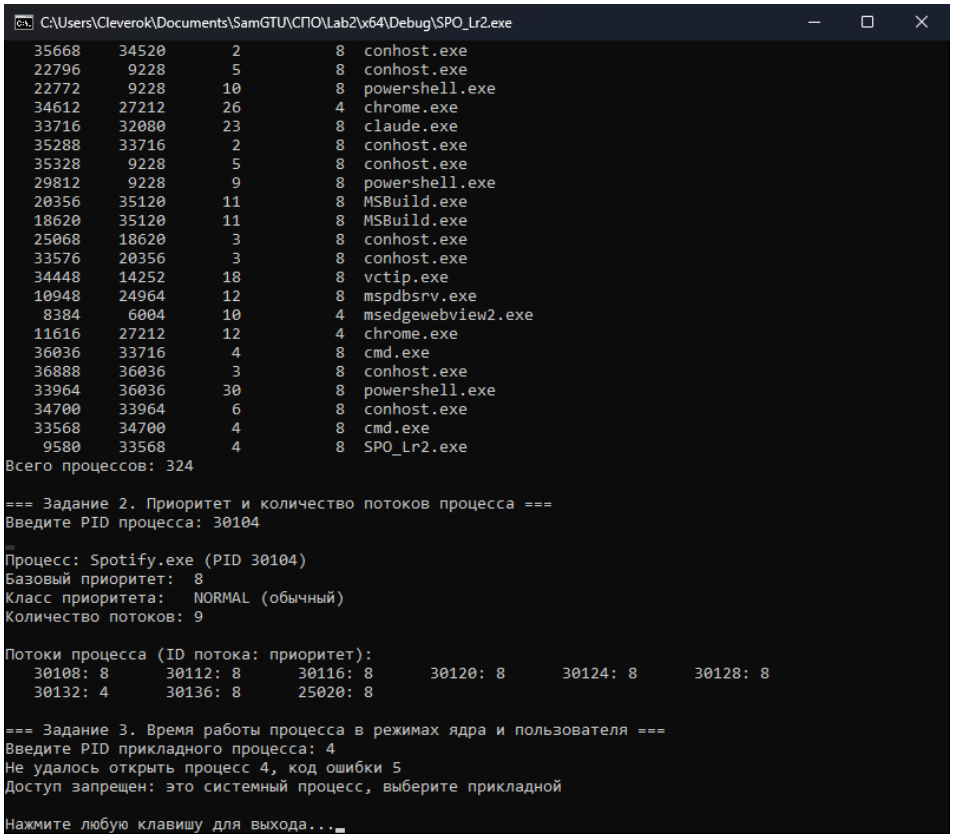


Рисунок 3 – Попытка получить время работы системного процесса (PID 4)

5. Индивидуальное задание 1. Загрузка процессора в процентах.

Алгоритм (программа SPO_Lr2_CpuLoad):

1) создать запрос функцией PdhOpenQuery;

2) сформировать функцией PdhMakeCounterPath пути к счетчикам \Processor(_Total)\% Processor Time и \Processor Information(_Total)\% Processor Utility и добавить их в запрос функцией PdhAddEnglishCounter;

3) выполнить первый (базовый) замер функцией PdhCollectQueryData;

4) 20 раз с интервалом 1 с (Sleep(1000)) выполнять замер и получать значения счетчиков функцией PdhGetFormattedCounterValue в формате PDH_FMT_DOUBLE, выводить их с текущим временем;

5) удалить счетчики и закрыть запрос.

Функция PdhAddEnglishCounter использована вместо PdhAddCounter из примера методических указаний: PdhAddCounter требует имена счетчиков на языке системы (в русской Windows – «Процессор», «% загруженности

процессора»), а PdhAddEnglishCounter принимает английские имена при любом языке Windows. Обе функции входят в библиотеку PDH.

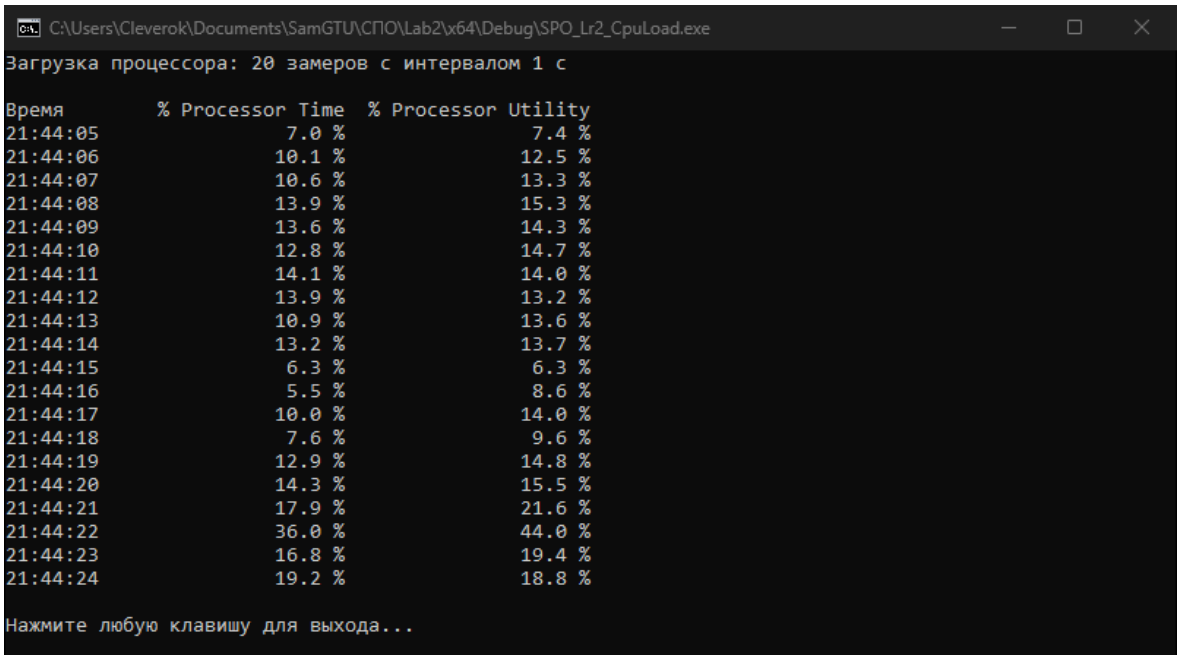


Рисунок 4 – Загрузка процессора (программа SPO_Lr2_CpuLoad)

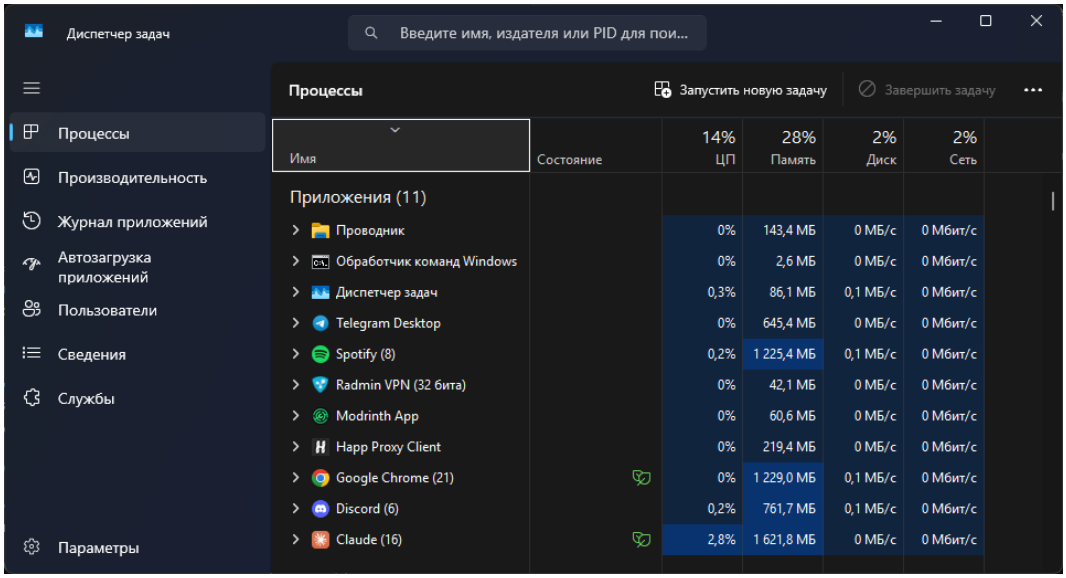


Рисунок 5 – Загрузка процессора в Диспетчере задач во время работы программы

Комментарий. За 20 замеров (рисунок 4) загрузка процессора составила от 5,5 до 36,0 % по счетчику % Processor Time и от 6,3 до 44,0 % по счетчику % Processor Utility; основную часть времени – 10–16 %. Снимок Диспетчера задач (рисунок 5) сделан во время работы программы, около 21:44:21; он показывает загрузку ЦП 14 %, а программа в 21:44:19 и 21:44:20 вывела 14,8 и 15,5 %. Значения совпадают с точностью до колебаний нагрузки между моментами замеров. Диспетчер задач в Windows 10/11 использует счетчик %

Processor Utility, учитывающий текущую частоту процессора; классический счетчик % Processor Time дает близкие значения.

6. Индивидуальное задание 2. Прикладные процессы, использующие более M байт оперативной памяти.

Алгоритм (программа SPO_Lr2_Memory):

1) ввести значение M (байт) с клавиатуры при запуске программы;
2) создать снимок процессов и перебрать его функциями Process32First / Process32Next;
3) пропустить процессы сеанса 0 (функция ProcessIdToSessionId) – это службы и системные процессы; остальные процессы считаются прикладными;
4) открыть процесс функцией OpenProcess и получить сведения о памяти функцией GetProcessMemoryInfo;
5) если объем рабочего набора (WorkingSetSize – объем оперативной памяти, занятой процессом) больше M, сохранить процесс в массив;

6) отсортировать массив по убыванию памяти (сортировка выбором) и вывести PID, имя и объем памяти в байтах и мегабайтах.

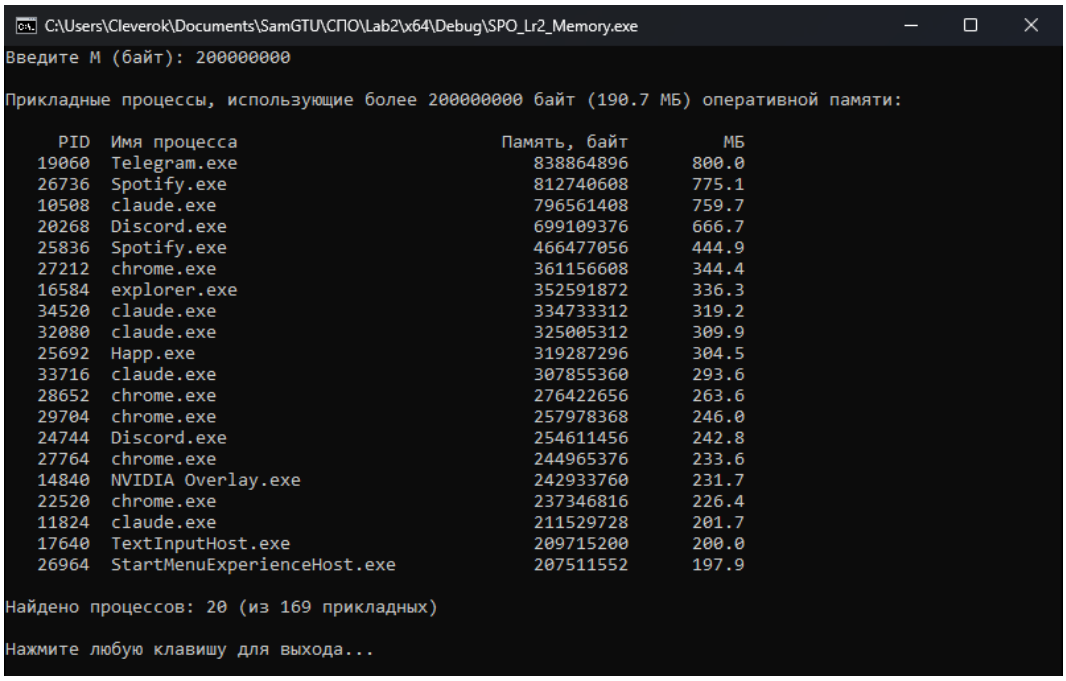


Рисунок 6 – Прикладные процессы, использующие более 200 000 000 байт (программа SPO_Lr2_Memory)

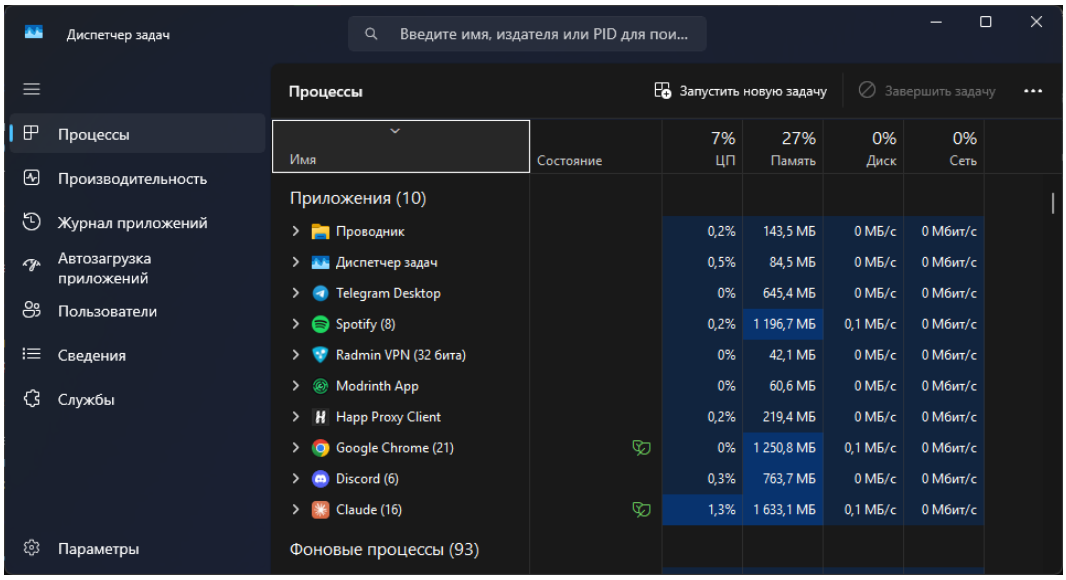


Рисунок 7 – Использование памяти приложениями в Диспетчере задач

Комментарий. При M = 200 000 000 байт найдено 20 процессов из 169 прикладных (рисунок 6); наибольший объем памяти используют Telegram.exe (800,0 МБ) и Spotify.exe (775,1 МБ). Результат совпадает с командой PowerShell Get-Process (свойство WorkingSet64). В Диспетчере задач (рисунок 7) на вкладке «Процессы» значения меньше: например, Telegram Desktop – 645,4 МБ. Это объясняется тем, что Диспетчер задач по умолчанию показывает активный частный рабочий набор (без страниц разделяемых

библиотек) и суммирует память всех процессов приложения, а программа выводит полный рабочий набор каждого процесса. Совпадающие значения дает столбец «Рабочий набор (память)» на вкладке «Подробности».

7. Тексты программ.

Листинг 1 – Общее задание (файл SPO_Lr2.cpp)

```
// Лабораторная работа № 2. Монитор процессов и потоков.
// Базовые задания: список процессов, приоритет и количество потоков процесса,
// время работы процесса в режиме ядра и в режиме пользователя.

#include <windows.h>
#include <tlhelp32.h>
#include <iostream>
#include <cstdio>
#include <conio.h>
#include <clocale>
using namespace std;

// Ожидание нажатия клавиши перед завершением программы
void WaitKey()
{
    wprintf_s(L"Нажмите любую клавишу для выхода...");
    _getch();
}

// Ввод идентификатора процесса с клавиатуры
DWORD ReadPid(const wchar_t* prompt)
{
    DWORD pid;
    while (true)
    {
        wprintf_s(L"%s", prompt);
        if (cin >> pid)
            return pid;
        wprintf_s(L"Ошибка ввода, повторите\n");
        cin.clear();
        cin.ignore(10000, '\n');
    }
}

// Задание 1. Вывод списка процессов, выполняющихся в системе
void PrintProcessList()
{
    HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnap == INVALID_HANDLE_VALUE)
    {
        wprintf_s(L"Ошибка CreateToolhelp32Snapshot, код %lu\n", GetLastError());
        return;
    }

    PROCESSENTRY32 pe;
    pe.dwSize = sizeof(PROCESSENTRY32);
    int count = 0;

    wprintf_s(L"%8s %8s %7s %10s  %s\n",
        L"PID", L"PPID", L"Потоков", L"Приоритет", L"Имя процесса");
    if (Process32First(hSnap, &pe))
    {
        do
        {
            wprintf_s(L"%8lu %8lu %7lu %10ld  %s\n", pe.th32ProcessID, pe.th32ParentProcessID,
                pe.cntThreads, pe.pcPriClassBase, pe.szExeFile);
            count++;
```

```
        } while (Process32Next(hSnap, &pe));
    }
    CloseHandle(hSnap);

    wprintf_s(L"Всего процессов: %d\n", count);
}

// Поиск процесса в снимке по идентификатору
bool FindProcess(DWORD pid, PROCESSENTRY32& pe)
{
    HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnap == INVALID_HANDLE_VALUE)
        return false;

    bool found = false;
    pe.dwSize = sizeof(PROCESSENTRY32);
    if (Process32First(hSnap, &pe))
    {
        do
        {
            if (pe.th32ProcessID == pid)
            {
                found = true;
                break;
            }
        } while (Process32Next(hSnap, &pe));
    }
    CloseHandle(hSnap);
    return found;
}

// Название класса приоритета процесса
const wchar_t* PriorityClassName(DWORD priorityClass)
{
    switch (priorityClass)
    {
    case IDLE_PRIORITY_CLASS:         return L"IDLE (низкий)";
    case BELOW_NORMAL_PRIORITY_CLASS: return L"BELOW_NORMAL (ниже среднего)";
    case NORMAL_PRIORITY_CLASS:       return L"NORMAL (обычный)";
    case ABOVE_NORMAL_PRIORITY_CLASS: return L"ABOVE_NORMAL (выше среднего)";
    case HIGH_PRIORITY_CLASS:         return L"HIGH (высокий)";
    case REALTIME_PRIORITY_CLASS:     return L"REALTIME (реального времени)";
    default:                          return L"неизвестен";
    }
}

// Задание 2. Приоритет и количество потоков выбранного процесса
void PrintPriorityAndThreads(DWORD pid)
{
    PROCESSENTRY32 pe;
    if (!FindProcess(pid, pe))
    {
        wprintf_s(L"Процесс с PID %lu не найден\n", pid);
        return;
    }

    wprintf_s(L"\nПроцесс: %s (PID %lu)\n", pe.szExeFile, pid);
    wprintf_s(L"Базовый приоритет:  %ld\n", pe.pcPriClassBase);

    HANDLE hProc = OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION, FALSE, pid);
    if (hProc != NULL)
    {
        wprintf_s(L"Класс приоритета:   %s\n", PriorityClassName(GetPriorityClass(hProc)));
        CloseHandle(hProc);
    }
    wprintf_s(L"Количество потоков: %lu\n", pe.cntThreads);
```

```
    // Список потоков процесса с их приоритетами
    HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPTHREAD, 0);
    if (hSnap == INVALID_HANDLE_VALUE)
        return;

    THREADENTRY32 te;
    te.dwSize = sizeof(THREADENTRY32);
    int n = 0;
    wprintf_s(L"\nПотоки процесса (ID потока: приоритет):\n");
    if (Thread32First(hSnap, &te))
    {
        do
        {
            if (te.th32OwnerProcessID == pid)
            {
                wprintf_s(L"%8lu: %-4ld", te.th32ThreadID, te.tpBasePri);
                if (++n % 6 == 0)
                    wprintf_s(L"\n");
            }
        } while (Thread32Next(hSnap, &te));
    }
    CloseHandle(hSnap);
    if (n % 6 != 0)
        wprintf_s(L"\n");
}

// Перевод времени FILETIME (интервалы по 100 нс) в миллисекунды
ULONGLONG FileTimeToMs(const FILETIME& ft)
{
    ULARGE_INTEGER t;
    t.LowPart = ft.dwLowDateTime;
    t.HighPart = ft.dwHighDateTime;
    return t.QuadPart / 10000;
}

// Вывод длительности в формате чч:мм:сс.мс
void PrintDuration(const wchar_t* title, const FILETIME& ft)
{
    ULONGLONG ms = FileTimeToMs(ft);
    wprintf_s(L"%s %02llu:%02llu:%02llu.%03llu (%.3f с)\n", title,
        ms / 3600000, ms / 60000 % 60, ms / 1000 % 60, ms % 1000, ms / 1000.0);
}

// Задание 3. Время работы процесса в режиме ядра и в режиме пользователя
void PrintProcessTimes(DWORD pid)
{
    HANDLE hProc = OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION, FALSE, pid);
    if (hProc == NULL)
    {
        DWORD err = GetLastError();
        wprintf_s(L"Не удалось открыть процесс %lu, код ошибки %lu\n", pid, err);
        if (err == ERROR_ACCESS_DENIED)
            wprintf_s(L"Доступ запрещен: это системный процесс, выберите прикладной\n");
        else if (err == ERROR_INVALID_PARAMETER)
            wprintf_s(L"Процесс с таким PID не существует\n");
        return;
    }

    FILETIME ftCreation, ftExit, ftKernel, ftUser;
    if (!GetProcessTimes(hProc, &ftCreation, &ftExit, &ftKernel, &ftUser))
    {
        wprintf_s(L"Ошибка GetProcessTimes, код %lu\n", GetLastError());
        CloseHandle(hProc);
        return;
    }
    CloseHandle(hProc);
```

```
    // Время запуска: UTC -> местное время -> SYSTEMTIME
    FILETIME ftLocal;
    SYSTEMTIME st;
    FileTimeToLocalFileTime(&ftCreation, &ftLocal);
    FileTimeToSystemTime(&ftLocal, &st);

    wprintf_s(L"\nВремя запуска процесса:        %02u.%02u.%04u %02u:%02u:%02u\n",
        st.wDay, st.wMonth, st.wYear, st.wHour, st.wMinute, st.wSecond);
    PrintDuration(L"Время в режиме ядра:          ", ftKernel);
    PrintDuration(L"Время в режиме пользователя:  ", ftUser);
}

int main()
{
    setlocale(LC_CTYPE, "rus");

    wprintf_s(L"=== Задание 1. Список процессов ===\n\n");
    PrintProcessList();

    wprintf_s(L"\n=== Задание 2. Приоритет и количество потоков процесса ===\n");
    PrintPriorityAndThreads(ReadPid(L"Введите PID процесса: "));

    wprintf_s(L"\n=== Задание 3. Время работы процесса в режимах ядра и пользователя ===\n");
    PrintProcessTimes(ReadPid(L"Введите PID прикладного процесса: "));

    wprintf_s(L"\n");
    WaitKey();
    return 0;
}
```

Листинг 2 – Индивидуальное задание 1 (файл SPO_Lr2_CpuLoad.cpp)

```
// Лабораторная работа № 2. Монитор процессов и потоков.
// Индивидуальное задание 1 (вариант 10): загрузка процессора в процентах.

#include <windows.h>
#include <pdh.h>
#include <conio.h>
#include <cstdio>
#include <clocale>
#pragma comment(lib, "pdh.lib")

const int COUNT = 20;   // количество замеров (с интервалом 1 с)

// Добавление счетчика в запрос. Путь формируется функцией PdhMakeCounterPath
// из английских имен объекта, экземпляра и счетчика; PdhAddEnglishCounter
// принимает английские имена независимо от языка Windows.
bool AddCounter(PDH_HQUERY hQuery, const wchar_t* object, const wchar_t* instance,
    const wchar_t* counter, PDH_HCOUNTER* hCounter)
{
    PDH_COUNTER_PATH_ELEMENTS cpe = {};
    cpe.szMachineName = NULL;
    cpe.szObjectName = const_cast<wchar_t*>(object);
    cpe.szInstanceName = const_cast<wchar_t*>(instance);
    cpe.szParentInstance = NULL;
    cpe.dwInstanceIndex = 0;
    cpe.szCounterName = const_cast<wchar_t*>(counter);

    wchar_t path[PDH_MAX_COUNTER_PATH];
    DWORD size = PDH_MAX_COUNTER_PATH;
    PDH_STATUS status = PdhMakeCounterPath(&cpe, path, &size, 0);
    if (status == ERROR_SUCCESS)
        status = PdhAddEnglishCounter(hQuery, path, 0, hCounter);
    if (status != ERROR_SUCCESS)
```

```
    {
        wprintf_s(L"Не удалось добавить счетчик %s, код 0x%08lX\n", path, status);
        return false;
    }
    return true;
}

// Ожидание нажатия клавиши перед завершением программы
void WaitKey()
{
    wprintf_s(L"Нажмите любую клавишу для выхода...");
    _getch();
}

int main()
{
    setlocale(LC_CTYPE, "rus");

    PDH_HQUERY hQuery;
    PDH_HCOUNTER hProcessorTime, hProcessorUtility;

    if (PdhOpenQuery(NULL, 0, &hQuery) != ERROR_SUCCESS)
    {
        wprintf_s(L"Ошибка PdhOpenQuery\n");
        WaitKey();
        return 1;
    }

    // \Processor(_Total)\% Processor Time – загруженность процессора (все ядра)
    // \Processor Information(_Total)\% Processor Utility – значение, которое
    // показывает Диспетчер задач (учитывает текущую частоту процессора)
    if (!AddCounter(hQuery, L"Processor", L"_Total",
            L"% Processor Time", &hProcessorTime) ||
        !AddCounter(hQuery, L"Processor Information", L"_Total",
            L"% Processor Utility", &hProcessorUtility))
    {
        PdhCloseQuery(hQuery);
        WaitKey();
        return 1;
    }

    // Первый замер – базовый: значение счетчика вычисляется по двум замерам
    PdhCollectQueryData(hQuery);

    wprintf_s(L"Загрузка процессора: %d замеров с интервалом 1 с\n\n", COUNT);
    wprintf_s(L"%-10s %18s %20s\n", L"Время", L"% Processor Time", L"% Processor Utility");

    for (int i = 0; i < COUNT; i++)
    {
        Sleep(1000);
        PdhCollectQueryData(hQuery);

        PDH_FMT_COUNTERVALUE time = {}, utility = {};
        PDH_STATUS s1 = PdhGetFormattedCounterValue(hProcessorTime,
            PDH_FMT_DOUBLE, NULL, &time);
        PDH_STATUS s2 = PdhGetFormattedCounterValue(hProcessorUtility,
            PDH_FMT_DOUBLE, NULL, &utility);
        if (s1 != ERROR_SUCCESS || s2 != ERROR_SUCCESS)
            continue;

        // Utility может превышать 100% при турбо-частоте,
        // Диспетчер задач ограничивает его значением 100%
        double util = utility.doubleValue > 100.0 ? 100.0 : utility.doubleValue;

        SYSTEMTIME st;
        GetLocalTime(&st);
```

```
        wprintf_s(L"%02u:%02u:%02u   %16.1f %% %18.1f %%\n",
            st.wHour, st.wMinute, st.wSecond, time.doubleValue, util);
    }
    PdhRemoveCounter(hProcessorTime);
    PdhRemoveCounter(hProcessorUtility);
    PdhCloseQuery(hQuery);

    wprintf_s(L"\n");
    WaitKey();
    return 0;
}
```

Листинг 3 – Индивидуальное задание 2 (файл SPO_Lr2_Memory.cpp)

```
// Лабораторная работа № 2. Монитор процессов и потоков.
// Индивидуальное задание 2 (вариант 10): список прикладных процессов,
// использующих более M байт оперативной памяти.

#include <windows.h>
#include <tlhelp32.h>
#include <psapi.h>
#include <iostream>
#include <cstdio>
#include <clocale>
#include <conio.h>
#pragma comment(lib, "psapi.lib")
using namespace std;

struct ProcInfo
{
    PROCESSENTRY32 pe;      // сведения о процессе из снимка
    SIZE_T workingSet;      // объем используемой оперативной памяти (рабочий набор), байт
};

const int MAX_PROC = 1024;
ProcInfo procList[MAX_PROC];    // процессы, удовлетворяющие условию отбора

// Ожидание нажатия клавиши перед завершением программы
void WaitKey()
{
    wprintf_s(L"Нажмите любую клавишу для выхода...");
    _getch();
}

int main()
{
    setlocale(LC_CTYPE, "rus");

    unsigned long long M;
    wprintf_s(L"Введите M (байт): ");
    if (!(cin >> M))
    {
        wprintf_s(L"Ошибка ввода M\n");
        WaitKey();
        return 1;
    }

    HANDLE hSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (hSnap == INVALID_HANDLE_VALUE)
    {
        wprintf_s(L"Ошибка CreateToolhelp32Snapshot, код %lu\n", GetLastError());
        WaitKey();
        return 1;
    }

    int count = 0;              // количество отобранных процессов
```

```
    int appCount = 0;           // количество просмотренных прикладных процессов

    PROCESSENTRY32 pe;
    pe.dwSize = sizeof(PROCESSENTRY32);
    if (Process32First(hSnap, &pe))
    {
        do
        {
            // Службы и системные процессы выполняются в сеансе 0,
            // прикладные процессы пользователя — в сеансах 1, 2, ...
            DWORD session;
            if (!ProcessIdToSessionId(pe.th32ProcessID, &session) || session == 0)
                continue;

            HANDLE hProc = OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION, FALSE,
                pe.th32ProcessID);
            if (hProc == NULL)
                continue;

            PROCESS_MEMORY_COUNTERS pmc;
            if (GetProcessMemoryInfo(hProc, &pmc, sizeof(pmc)))
            {
                appCount++;
                if (pmc.WorkingSetSize > M && count < MAX_PROC)
                {
                    procList[count].pe = pe;
                    procList[count].workingSet = pmc.WorkingSetSize;
                    count++;
                }
            }
            CloseHandle(hProc);
        } while (Process32Next(hSnap, &pe));
    }
    CloseHandle(hSnap);

    // Сортировка выбором по убыванию используемой памяти
    for (int i = 0; i < count - 1; i++)
    {
        int maxIndex = i;
        for (int j = i + 1; j < count; j++)
        {
            if (procList[j].workingSet > procList[maxIndex].workingSet)
                maxIndex = j;
        }
        if (maxIndex != i)
        {
            ProcInfo tmp = procList[i];
            procList[i] = procList[maxIndex];
            procList[maxIndex] = tmp;
        }
    }

    wprintf_s(L"\nПрикладные процессы, использующие более %llu байт (%.1f МБ) "
        L"оперативной памяти:\n\n", M, M / 1048576.0);
    wprintf_s(L"%8s  %-32s %16s %10s\n", L"PID", L"Имя процесса", L"Память, байт", L"МБ");
    for (int i = 0; i < count; i++)
    {
        wprintf_s(L"%8lu  %-32s %16zu %10.1f\n",
            procList[i].pe.th32ProcessID, procList[i].pe.szExeFile,
            procList[i].workingSet, procList[i].workingSet / 1048576.0);
    }
    wprintf_s(L"\nНайдено процессов: %d (из %d прикладных)\n\n", count, appCount);

    WaitKey();
    return 0;
}
```

Вывод: в ходе работы с помощью функций Tool Help Library (CreateToolhelp32Snapshot, Process32First/Next, Thread32First/Next) получен список процессов системы, сведения о приоритете и потоках выбранного процесса; функцией GetProcessTimes определено время работы процесса в режимах ядра и пользователя; с помощью счетчиков производительности PDH измерена загрузка процессора; функцией GetProcessMemoryInfo из библиотеки PSAPI определен объем оперативной памяти, используемой прикладными процессами. Результаты совпадают с данными Диспетчера задач и PowerShell с учетом различий в определении показателей. Для части системных процессов функция OpenProcess возвращает ошибку «доступ запрещен», поэтому сведения о времени и памяти получены для прикладных процессов.